

DevSight AI — Project Master Guide

Autonomous Cloud Observability, AI Root Cause Diagnosis & Viva Preparation

Repository: github.com/LalithKrishna12/DevSight-Real-Time	Architecture: Microservices Observability Monolith
Stack: React 18, TypeScript, Node/Express, Prisma, PostgreSQL	AI Engine: Claude 3.7 + Heuristic Fallback Engine

1. Executive Summary & Problem Statement

Modern cloud environments consist of dozens of distributed microservices. When incidents or slowdowns occur, engineering teams are bombarded with hundreds of metric graphs and cryptic error logs. Identifying the true root cause during high-severity outages often takes hours of manual correlation.

DevSight AI is an autonomous observability platform that actively diagnoses failures. Instead of just displaying graphs, it detects statistical metric anomalies, correlates telemetry snapshots with error logs, and delivers an actionable, plain-English root cause explanation with remediation steps in under three seconds.

2. System Architecture & Core Components

Component	Technology	Responsibility
Frontend UI	React 18, TypeScript, Vite	Low-glare dark ops dashboard, real-time auto-refresh, incident drill-down, live log stream & 1-click anomaly simulator.
API & Backend	Node.js, Express.js	REST endpoints for auth, metrics ingestion, log query, service management, and incident lifecycles.
Database Layer	PostgreSQL, Prisma ORM	Type-safe relational storage for Multi-tenant Orgs, Users, Monitored Services, Metrics, Logs, and Incidents.
Anomaly Detector	Statistical 3σ Rolling Baseline	Computes rolling mean and standard deviation over last 200 samples; flags metrics deviating by $\geq 3\sigma$.
AI Root Cause Engine	Claude 3.7 + Heuristic Core	Correlates anomalous metrics and error logs into structured diagnosis: What Failed, Why, Impact, Suggested Fix.
Alert Engine	Multi-Channel Dispatcher	Instant webhook formatting and delivery across Slack, Email (SMTP), and Microsoft Teams.

3. Autonomous Incident Pipeline Workflow

- Step 1 — Ingestion:** Microservice agents POST telemetry data to `/api/metrics/ingest`.
- Step 2 — Statistical Evaluation:** `recordMetricAndCheck` recalculates rolling baseline. If deviation $\geq 3\sigma$, an anomaly is flagged.
- Step 3 — Context Snapshot:** `triggerIncidentAnalysis` pulls the latest 20 metric points and 30 logs for that service.

- **Step 4 — AI Root Cause Synthesis:** AI correlates logs and metrics to generate a plain-English explanation.
- **Step 5 — Incident Persistence & Alerts:** Incident is saved with status OPEN; alerts are dispatched to Slack/Email/Teams.
- **Step 6 — Live Dashboard Update:** Service status turns DEGRADED and the incident report appears on the unified dashboard.

4. Comprehensive Viva & Technical Interview Questions

Prepare these exact answers for technical interviews and project viva defense:

Q1: Why did you choose statistical anomaly detection (3-sigma) over fixed threshold alerting?

Answer: Fixed thresholds (e.g., latency > 500ms) produce rampant false positives because microservices have varying normal baselines (an Auth service averages 50ms, while a Report generator takes 2000ms). By calculating rolling mean (μ) and standard deviation (σ), the system autonomously learns each service's baseline and alerts only on true anomalies ($\geq 3\sigma$).

Q2: How does the system guarantee zero downtime if the external LLM API is unavailable?

Answer: I built a resilient dual-engine architecture: when an Anthropic API key is configured, it leverages Claude 3.7 Sonnet for advanced synthesis. If the key is missing or the external API times out, it seamlessly falls back to an internal Heuristic Correlation Engine that parses error codes, stack traces, and deviation scores to output complete root-cause diagnostics without crashing.

Q3: Explain the role and advantages of Prisma ORM in this architecture.

Answer: Prisma provides end-to-end type safety, auto-generated migrations, and structured relationship queries. It maps Organizations, Users, Services, Metrics, Logs, and Incidents with referential integrity and indexes time-series fields (e.g. `@@index([serviceld, timestamp])`) for sub-millisecond query performance.

Q4: How does DevSight AI ensure secure Multi-Tenancy and Access Control?

Answer: Multi-tenancy is enforced at the database layer where all queries scope to `req.user.organizationId` verified via JWT tokens. Role-Based Access Control (RBAC) guards sensitive management actions (such as adding or deleting services) restricting them strictly to ADMIN and DEVOPS_ENGINEER roles.

Q5: How does the Log Analyzer feature provide value to engineers?

Answer: It provides dual functionality: 1) A live, searchable log browser with level-based filtering (INFO/WARN/ERROR), and 2) An on-demand AI Explainer where engineers can click 'Explain with AI' on any live error or paste custom stack traces to get an instant plain-English translation and remediation plan.

Q6: How would you scale this architecture to ingest millions of metrics per second?

Answer: I would introduce a distributed message broker (Apache Kafka or Redis Streams) in front of the ingest endpoint to buffer bursts, migrate metric storage to a specialized time-series database (TimescaleDB or ClickHouse), and decouple backend modules into dedicated microservices (Auth, Ingest, AI Analysis, Alerting).

5. Business Model & Startup Strategy

Market Opportunity: The global observability market exceeds \$25 Billion. Traditional tools like Datadog and Dynatrace charge tens of thousands of dollars just to show graphs. DevSight AI's competitive edge is **autonomous diagnosis**—answering 'WHY' an incident happened rather than just showing charts.

Plan Tier	Price	Target Customer	Features Included
Starter	\$29 / mo	Early startups & indie developers	Up to 5 services, 7-day metric retention, Slack alerts, 200 AI root cause reports/mo.

Professional	\$199 / mo	Growth-stage SaaS companies	Up to 25 services, 30-day retention, Slack + Teams + Email alerts, unlimited AI reports, full RBAC.
Enterprise	Custom (\$999+/mo)	Large scale engineering orgs	Unlimited services, 1-year retention, Prometheus/OTel connectors, SSO/SAML, dedicated LLM deployment.

6. Key Business Metrics & ROI for Customers

- ✓ **70% Reduction in MTTR (Mean Time to Resolution):** Outage root causes diagnosed in 3 seconds instead of 45-minute triaging calls.
- ✓ **Eliminates Alert Fatigue:** Statistical 3σ anomaly detection filters out background noise, alerting only when genuine failures occur.
- ✓ **Zero On-Call Ramp-Up Time:** Junior developers can resolve incidents immediately because root causes and fixes are explained in plain English.